AEGIS: Efficiency-Guided Multitask Distillation for Joint Vulnerability and Technical-Debt Detection

Awaiz Ahmed
Affiliation (to be completed)
email (to be completed)

***Abstract*—The NLBSE'27 tool competition ranks single multitask models that detect weaknesses and self-admitted technical debt (SATD) in C/C++ functions by a score that combines mean F1 with inference runtime and GFLOPs. AEGIS treats this score as the design objective. An audit of the data shows that the SATD label is a keyword annotation whose evidence is partly missing from the released functions, and that the official pipeline wastes most of its computation on padding. AEGIS shortens the input with whitespace normalization and a compact tokenizer, pads batches dynamically, chooses encoder depth by the projected score, and distills a 12-layer UniXcoder teacher into a 3-layer student with per-task thresholds folded into the model. On the public test split, AEGIS improves mean F1 from 0.910 to 0.931 while using 13× fewer GFLOPs than the CodeBERT baseline, raising the competition score from 0.754 to 0.944 on the same GPU.**

***Index Terms*—vulnerability detection, self-admitted technical debt, multitask learning, knowledge distillation, efficient inference**

I. Introduction

The NLBSE'27 tool competition asks for a single model that reads a C/C++ function and predicts two labels: whether the function is associated with a weakness or vulnerability, and whether it carries a self-admitted technical debt (SATD) annotation [1]. Submissions are ranked by a score that weights the mean of the two F1 scores by 0.6 and the inference runtime and GFLOPs, measured with `torch.profiler` over the whole test split on a Colab T4, by 0.2 each. The CodeBERT [2] baseline of the organizers reaches a mean F1 of 0.910 but consumes 2.67M of the 3M-GFLOPs budget, so its score is 0.729.

AEGIS treats the ranking score itself as the design objective. Our contributions are: (i) an analysis of the task and the score: the SATD label is a keyword annotation whose evidence is partly missing from the input, and the score implies fixed exchange rates between F1, runtime and FLOPs that an analytic cost model predicts to within 0.04%; (ii) length-sorted dynamic padding, which alone raises the score of the unchanged baseline checkpoint from 0.754 to 0.862, and input compaction, which shortens inputs by a third; (iii) a 3-layer multitask student distilled from a 12-layer UniXcoder [3] teacher, whose depth was chosen by the score rather than by F1. It is exported as a standard Hugging Face model that loads in the official notebook.

II. Task, Data, and Scoring

Data. The dataset derives from MADE-WIC [4], which merges OSPR (Linux, Chromium, Firefox), Devign [5] and Big-Vul [6]. The weakness label is the logical OR of the Devign, Big-Vul and CWE-heuristic (W [7]) annotations. The SATD label is MAT, the "Matches task Annotation Tags" heuristic [8]: a function is positive if a comment contains TODO, FIXME, XXX or HACK. Functions are clang-formatted, split 70/10/20 with joint stratification (214,586 training and 30,655 public test functions; 61,311 hidden), and only the `Function` text may be used as input. Weakness prevalence is 32.8%, MAT prevalence 1.65%.

Audit findings. (1) A word-boundary match of the four tags inside comments reaches MAT precision 1.000, recall 0.877 and F1 0.935 on the public test split. The misses are MAT positives with no tag anywhere in the released text. The organizers' curation script loads the comment *above* each function but does not release it, which most likely explains them. Roughly 12% of MAT positives therefore cannot be recovered from the input, which caps MAT F1 near 0.935. (2) Weakness correlates with length: its prevalence rises from 8.5% to 60.3% across length quartiles. (3) 18% of test functions have an identifier-abstracted clone in the training split, whose weakness label agrees in 98% of cases. Random splits are known to inflate vulnerability-detection scores [9], [10].

Scoring. With $\bar{F}_1$ the mean F1, t the runtime in seconds and G the GFLOPs (both totals over the split, averaged over two timed forward passes),

$$S = 0.6\,\bar{F}_1 + 0.2\max\!\left(\tfrac{1000-t}{1000}, 0\right) + 0.2\max\!\left(\tfrac{3\cdot10^6-G}{3\cdot10^6}, 0\right). \quad (1)$$

One F1 point (0.01) is thus worth 0.006 score, as much as 30 s of runtime or 90,000 GFLOPs. `torch.profiler` attributes FLOPs only to matrix multiplications, so for an encoder with L layers, hidden size d and feed-forward size d_{ff},

$$G \approx 10^{-9} \sum_b B_b W_b\, L\,(8d^2 + 4d\,d_{ff}), \quad (2)$$

where batch b has B_b sequences padded to width W_b; fused attention is not counted. This estimate matches every profiler measurement in this paper within 0.04%. The official notebook pads the whole split to 512 tokens, so only 43% of processed positions are real tokens.

III. The AEGIS Approach

Input compaction. clang-format indentation becomes long runs of whitespace tokens. We collapse every whitespace run to a single space and use UniXcoder's tokenizer. This shortens the mean input from 235 (CodeBERT, raw) to 154 tokens and the share of truncated functions from 10.6% to 0.5% at 512 tokens. Comments are kept, since they carry the SATD tags.

Length-sorted dynamic padding. At inference we sort functions by length, form batches of 600 and pad each batch only to its own maximum, then restore the original order. Predictions are unchanged; only padding work disappears. The competition allows any test-time preprocessing except sampling.

Score-guided depth. The student keeps the embeddings and the first k transformer layers of the encoder, consistent with the finding that early layers of code encoders suffice for many classification tasks [11]. One shared encoder feeds the standard two-output classification head, so the checkpoint is a plain `RobertaForSequenceClassification`. We choose k by the projected competition score on internal validation, not by F1.

Multitask distillation. A 12-layer UniXcoder teacher is fine-tuned on the same data. The student, initialized from the teacher's first k layers, minimizes

$$\mathcal{L} = \frac{\alpha}{2} \sum_t \mathrm{BCE}(z_t, y_t) + \frac{1-\alpha}{2} T^2 \sum_t \mathrm{BCE}\big(z_t/T, \sigma(z_t^{\mathrm{T}}/T)\big), \tag{3}$$

over the tasks $t \in \{\text{weakness}, \text{MAT}\}$, with student logits z_t, teacher logits z_t^{T}, $\alpha = 0.5$ and $T = 2$ [12].

Per-task thresholds. MAT is rare (1.65%), so a fixed 0.5 threshold is unlikely to maximize its F1 [13]. For each task we pick the F1-maximizing threshold on internal validation and store it in the model configuration.

IV. Experimental Setup

We hold out a stratified 10% of the official training split as internal validation (21,243 functions; exact duplicates of training functions are moved to the fit side). Model selection, early stopping and threshold tuning use only this split. The public test split is used once per reported model. All models are trained with AdamW (learning rate $2 \cdot 10^{-5}$ for the teacher, $5 \cdot 10^{-5}$ for students), batch size 32, linear warmup (6%) and decay, bf16, length-grouped batches and seed 42 unless stated otherwise. The teacher trains for 3 epochs and students for 4. Inference cost is measured with the official measurement loop (two timed forward passes per batch under fp16 autocast and `torch.profiler`), re-implemented and validated against the official notebook (identical GFLOPs and predictions for the baseline), on an RTX 5080 Laptop GPU. All rows of Table I were measured in one session in the same power mode; the ablations were measured earlier in a lower power mode. Runtimes on the T4 used for the official ranking differ; GFLOPs do not depend on the hardware.

V. Results

Main results. Table I reports every model on the public test split with the official measurement loop. AEGIS reaches a mean F1 of 0.931 (+0.021 over the baseline) while processing 13× fewer GFLOPs and running 11× faster than the baseline on the same GPU, which raises the score from 0.754 to 0.944. The F1 gain comes from MAT (+0.049; recall 0.80→0.88); on weakness AEGIS is 0.007 below the 12-layer CodeBERT. Length-sorted dynamic padding alone, applied to the unchanged baseline checkpoint, keeps its predictions and lifts its score to 0.862. The 12-layer teacher is the most accurate model (0.938) but scores 0.904 because of its cost.

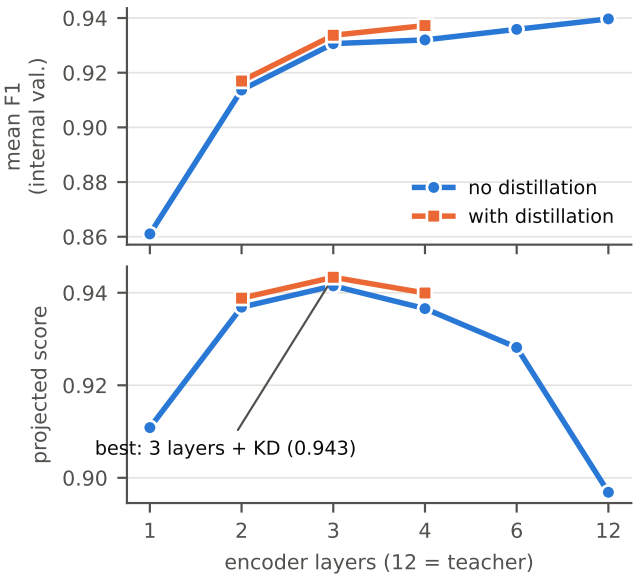


Fig. 1. Encoder depth vs. internal-validation mean F1 (top) and projected competition score (bottom). Points at 3 layers average three seeds.

Depth and distillation. Mean F1 grows with depth, but the projected score peaks at three layers (Fig. 1): from one to two layers weakness F1 rises by almost 0.1, while beyond three layers the added cost outweighs the remaining gains. Distillation helps at every depth we tried (Table II). Over three seeds at three layers it raises internal-validation mean F1 from 0.9306±0.0008 to 0.9337±0.0002 (Welch $p = 0.018$); the gain is entirely in weakness F1 (+0.0065, $p = 0.0013$), since MAT is already at its ceiling. Threshold calibration adds about 0.002 mean F1, raw (non-normalized) input trades weakness F1 for MAT F1 at 17% more cost, and a narrow encoder trained from scratch stays at 0.859, which suggests that code pretraining matters more than width at this budget.

Robustness to the evaluation procedure. It is not specified whether the organizers run participants' inference code or the published notebook unchanged. The submitted model is trained on a seeded 50/50 mix of raw and whitespace-normalized functions, and its thresholds are folded into the output bias ($b_t \leftarrow b_t - \mathrm{logit}(\tau_t)$), which reproduces the tuned decisions exactly at a 0.5 threshold. Run through the unmodified notebook (raw input, padding to 512), AEGIS still reaches a mean F1 of 0.927, at 667k GFLOPs and a score of 0.908.

TABLE I

PUBLIC-TEST RESULTS WITH THE OFFICIAL MEASUREMENT LOOP (TWO TIMED FORWARD PASSES PER BATCH, FP16, `torch.profiler`). RUNTIME AND GFLOPS ARE TOTALS OVER THE 30,655 FUNCTIONS. EXCEPT FOR THE FIRST ROW (VALUES PRINTED IN THE OFFICIAL NOTEBOOK, COLAB T4), ALL ROWS WERE MEASURED ON ONE RTX 5080 LAPTOP GPU IN THE SAME SESSION; RUNTIMES ARE HARDWARE-DEPENDENT, GFLOPS ARE NOT. THE LAST ROW RUNS AEGIS THROUGH THE UNMODIFIED OFFICIAL NOTEBOOK PROCEDURE (RAW INPUT, PADDING TO 512, THRESHOLD 0.5).

Model	Layers	Weak. F1	MAT F1	Mean F1	Runtime (s)	GFLOPs	Score
CodeBERT baseline, organizers (T4)	12	0.939	0.881	0.910	198.9	2,667k	0.729
CodeBERT baseline, official loop	12	0.939	0.881	0.910	71.0	2,666k	0.754
+ sorted dynamic padding	12	0.939	0.881	0.910	30.5	1,172k	0.862
UniXcoder teacher	12	0.949	0.928	0.938	21.5	824k	0.904
Student, no distillation	3	0.929	0.927	0.928	6.0	206k	0.942
AEGIS	3	0.932	0.931	0.931	6.2	206k	0.944
AEGIS in the unmodified notebook	3	0.935	0.918	0.927	19.0	667k	0.908

TABLE II

INTERNAL-VALIDATION ABLATIONS. F1 USES VALIDATION-TUNED THRESHOLDS; COST (GFLOPS) IS MEASURED WITH THE OFFICIAL LOOP ON INTERNAL VALIDATION AND THE SCORE PROJECTS IT TO THE PUBLIC-TEST SIZE. †MEAN OF THREE SEEDS (STANDARD DEVIATION OF MEAN F1: 0.0008 AND 0.0002). COSTS WERE MEASURED IN A LOWER GPU POWER MODE THAN TABLE I.

Model	Weak. F1	MAT F1	Mean F1	GFLOPs	Score
1 layer	0.805	0.917	0.861	48k	0.911
2 layers	0.901	0.927	0.914	97k	0.937
3 layers†	0.930	0.931	0.931	145k	0.941
4 layers	0.933	0.931	0.932	193k	0.937
6 layers	0.942	0.930	0.936	290k	0.928
12 layers (teacher)	0.948	0.931	0.940	580k	0.897
2 layers + KD	0.908	0.925	0.917	97k	0.939
3 layers + KD†	0.937	0.931	0.934	145k	0.943
4 layers + KD	0.943	0.931	0.937	193k	0.940
2L + KD, pretrained init	0.907	0.928	0.917	97k	0.939
3L + KD, 6 epochs	0.938	0.930	0.934	145k	0.943
3L + KD, raw input	0.942	0.924	0.933	169k	0.940
3L + KD, mixed input (final)	0.935	0.931	0.933	145k	0.943
4×256 from scratch	0.803	0.916	0.859	22k	0.913

Error analysis. On the public test split, 61 of the 62 MAT false negatives contain no SATD tag in the released function, and on functions with a tag MAT F1 is 0.999. Weakness errors concentrate on short functions (F1 0.870 up to 64 tokens vs. 0.952 for 257–512 tokens) and vary strongly by project: F1 is 0.955 on the Linux kernel (76% of the test split) but 0.777 on Chrome, and 0.869 and 0.707 on the two Devign projects, FFmpeg and qemu. Functions with a structural clone in the training split reach 0.959 vs. 0.930 for the rest.

VI. LIMITATIONS

Labels. Both labels are partly heuristic: MAT is a keyword rule, W is a set of CWE-derived patterns, and Big-Vul and Devign labels are known to be noisy [9], [10]. High F1 on this benchmark measures agreement with these annotations, not the ability to find real vulnerabilities, and AEGIS should not be used as a security tool on that basis. **Generalization.** The split is random and stratified, and 18% of test functions have a structural clone in training, so our scores likely overestimate performance on unseen projects or on later code. Weakness prevalence also depends strongly on length and project, which a model may exploit as shortcuts. **Measurement.** Runtime depends on the GPU, driver and power mode; our runtimes come from a laptop GPU, while the ranking uses a T4. GFLOPs follow the profiler's convention and omit fused attention, which our analytic model quantifies separately. Several scoring details (number of repetitions, constants for the larger hidden split) are ambiguous in the official material; we follow the published notebook code. **Scope.** We explored one encoder family and a single distillation objective, and ran seed repeats only for the final configuration and its no-distillation counterpart.

VII. CONCLUSION

Under an efficiency-weighted ranking, the largest gains for NLBSE'27 came from reading the score: shrinking what the model processes (whitespace normalization, a compact tokenizer, dynamic padding) and choosing depth by the projected score. Multitask distillation then recovers about a third of the F1 gap to the 12-layer teacher at a quarter of its cost. MAT is effectively solved up to the information missing from the released functions; remaining errors are in weakness detection, especially for short functions. Code, configurations, experiment logs and the trained model are available at https://github.com/4waiz/AEGIS.

ACKNOWLEDGMENT

The author used an AI assistant (Claude, Anthropic) to implement the experiment code, run the experiments, analyze the results and draft this manuscript. All numbers are generated by the released scripts from the logged experiment outputs; the author is responsible for the content of the paper.

REFERENCES

[1] M. Mock, T. Borsani, B. Russo, F. Santos, J. Penney, P. Valenzuela-Toledo, T. Kehrer, and S. Panichella, "The NLBSE'27 tool competition and challenge," Proceedings of the 6th International Workshop on Natural Language-based Software Engineering (NLBSE'27), to appear, 2027.

[2] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 1536–1547.

[3] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, "UniXcoder: Unified cross-modal pre-training for code representation," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2022, pp. 7212–7225.

[4] M. Mock, J. Melegati, M. Kretschmann, N. E. Diaz Ferreyra, and B. Russo, "MADE-WIC: Multiple annotated datasets for exploring weaknesses in code," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, 2024, pp. 2346–2349.

[5] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[6] J. Fan, Y. Li, S. Wang, and T. N. Nguyen, "A C/C++ code vulnerability dataset with code changes and CVE summaries," in *Proceedings of the 17th International Conference on Mining Software Repositories (MSR)*, 2020, pp. 508–512.

[7] B. Russo, M. Camilli, and M. Mock, "WeakSATD: Detecting weak self-admitted technical debt," in *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*, 2022, pp. 448–453.

[8] Z. Guo, S. Liu, J. Liu, Y. Li, L. Chen, H. Lu, and Y. Zhou, "How far have we progressed in identifying self-admitted technical debts? A comprehensive empirical study," *ACM Transactions on Software Engineering and Methodology*, vol. 30, no. 4, pp. 1–56, 2021.

[9] R. Croft, M. A. Babar, and M. M. Kholoosi, "Data quality for software vulnerability datasets," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 121–133.

[10] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, and Y. Chen, "Vulnerability detection with code language models: How far are we?" in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2025, pp. 1729–1741.

[11] A. Grishina, M. Hort, and L. Moonen, "The EarlyBIRD catches the bug: On exploiting early layers of encoder models for more efficient code classification," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2023, pp. 895–907.

[12] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," in *NIPS 2014 Deep Learning Workshop*, 2015, arXiv:1503.02531.

[13] Z. C. Lipton, C. Elkan, and B. Naryanaswamy, "Optimal thresholding of classifiers to maximize F1 measure," in *Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*, ser. Lecture Notes in Computer Science. Springer, 2014, pp. 225–239.